HYBRID ATTESTATION AND BEHAVIORAL FRAMEWORK FOR REAL-TIME BYOVD DETECTION IN ENDPOINT PROTECTION PLATFORMS

A PROJECT REPORT

Submitted by

KARTHIK R **(2116231901017)**

JANIT B **(2116231901013)**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**HYBRID ATTESTATION AND BEHAVIORAL FRAMEWORK FOR REAL-TIME BYOVD DETECTION IN ENDPOINT PROTECTION PLATFORMS**" is the bonafide work of **KARTHIK R (2116231901017), JANIT B (2116231901013)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

SIGNATURE

Dr. Sivakumar D

SUPERVISOR

ASSISTANT PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

Bring Your Own Vulnerable Driver (BYOVD) is a technique where an attacker loads a legitimate but exploitable kernel driver to gain elevated privileges on Windows systems. To counter this, modern Endpoint Detection and Response (EDR) agents deploy telemetry gathering in kernel space. However, attackers increasingly use BYOVD primitives to terminate or blind these user-mode analysis services. This paper presents a comprehensive, lightweight detection and self-protection framework. We implement the core logic in kernel mode as a hybrid Windows Driver Model (WDM) and Minifilter driver that intercepts driver loads in real-time while actively stripping dangerous access rights (such as termination and virtual memory writing) from any handles opened against the associated user-mode service. The user-mode engine, developed in C# on the .NET framework, receives telemetry via a Filter Manager communication port, performs offline hash attestation against a curated blocklist of known vulnerable drivers, and exposes a real-time web dashboard for analyst decision-making. By delegating disk I/O and cryptographic hashing to user-mode while maintaining strict self-defense in Ring 0, the architecture ensures kernel stability, prevents EDR blinding, and imposes minimal execution overhead on the critical image-load path.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E**., for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D**., for her enthusiastic motivation which inspired us in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S**., for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D**., and **Dr. P. Tamilselvi Ph.D**., **Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Dr. Sivakumar D, Assistant Professor**, for his encouragement and guidance throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG)**, for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project.

KARTHIK R 2116231901017
JANIT B 2116231901013

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	ACKNOWLEDGEMENT	**iv**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	1
	1.4 SCOPE	2
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	2
	1.6 SDG ALIGNMENT	2
2	**LITERATURE REVIEW**	**4**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	4
	2.2 RESEARCH GAPS	4
	2.3 PROPOSED SYSTEM	5
3	**SYSTEM DESIGN**	**6**
	3.1 SYSTEM ARCHITECTURE	6
	3.2 MODULE DESIGN	6
	3.3 DATA FLOW / UML DIAGRAMS	7
	3.4 THREAT MODEL AND ATTACK SURFACE	7
	3.5 SECURITY CONTROLS	8
4	**SYSTEM DESCRIPTION**	**9**
	4.1 TECHNOLOGY STACK AND TEST ENVIRONMENT	9
	4.2 MODULE IMPLEMENTATION	9
	4.3 RESULTS AND DISCUSSION	10
	4.4 IMPLEMENTATION VALIDATION	12
5	**CONCLUSION**	**13**
	FUTURE WORK	13
	REFERENCES	**15**
	APPENDIX	**16**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT	3
2.1	RELEVANT BYOVD APPROACHES	4
3.1	PROJECT MODULES	7
3.2	SECURITY CONTROLS AGAINST IDENTIFIED THREATS	8
4.1	IMPLEMENTATION VALIDATION RESULTS	10

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	HYBRID BYOVD DETECTION ARCHITECTURE	6
3.2	PROJECT MODULE DECOMPOSITION	6
3.3	DATA FLOW BETWEEN KERNEL AND USER MODE	7
3.4	THREAT MODEL AND DEFENCE SURFACE	7
4.1	IMPLEMENTATION VALIDATION WORKFLOW	12

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Windows kernel drivers execute with high privilege and can interact directly with operating-system resources. Bring Your Own Vulnerable Driver (BYOVD) attacks abuse legitimate, signed but vulnerable drivers to obtain kernel-level capabilities that can be used to interfere with endpoint security. A valid signature establishes the origin of a driver, but it does not by itself establish that the driver is safe to use.

A practical BYOVD defense therefore needs visibility at the kernel level. This project uses a lightweight driver to observe kernel-mode driver image loads and to protect the project's own user-mode analysis service from common user-mode termination and memory-access operations. The expensive work of reading the driver file and calculating its SHA-256 hash is delegated to user mode, reducing work on the sensitive kernel notification path.

1.2 PROBLEM STATEMENT

The central problem is identifying and responding to vulnerable driver activity without relying only on digital signatures or a user-mode monitor that can itself be terminated by an attacker. The prototype addresses four practical concerns:

- **Signed-driver trust gap:** a signed driver may still contain exploitable functionality.
- **Real-time visibility:** driver-load activity must be observed from a privileged kernel component.
- **Self-protection:** the monitoring service must resist administrative user-mode termination attempts.
- **Operational safety:** kernel callbacks must not wait for a dashboard or human interaction.

1.3 OBJECTIVES

- Develop a lightweight Ring-0 kernel driver using Windows Driver Kit (WDK) APIs.

- Detect kernel-mode driver image loads using PsSetLoadImageNotifyRoutine.
- Transfer driver-load events to a C# .NET service using a Filter Manager communication port.
- Compute SHA-256 hashes and compare them against a LOLDrivers-derived vulnerable-driver database.
- Protect the monitoring service from unauthorized termination and memory-write access using ObRegisterCallbacks.
- Provide a web dashboard for event visibility, verdicts, logs and administrative decisions.

1.4 SCOPE

The current scope is Windows x64 driver-load monitoring, SHA-256 attestation, user-mode event processing, service self-protection and dashboard-oriented reporting. The implementation was tested in an isolated virtual-machine environment. The project does not claim pre-load driver blocking through PsSetLoadImageNotifyRoutine, nor does it claim resistance to kernel-originated handles created by a separate vulnerable driver.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

- **Kernel safety:** the driver must register and remove callbacks correctly and unload without crashing the test system.
- **Least privilege:** the communication port is restricted to trusted administrative/system clients.
- **Non-blocking notification path:** driver-to-service messaging uses a zero timeout so the image-load callback does not wait on user mode.
- **Service integrity:** dangerous user-mode process rights such as PROCESS_ TERMINATE and PROCESS_VM_WRITE are stripped for handles targeting the protected service.
- **Auditability:** driver path, hash and verdict are recorded for later inspection.

1.6 SDG ALIGNMENT

The project aligns with SDG 9 by developing security-monitoring infrastructure and lightweight kernel-level security mechanisms. It also supports SDG 16 by improving trustworthy digital infrastructure through auditable security controls and defensive monitoring.

Table 1.1 SDG Alignment

S. No.	SDG No. & Goal	Project Contribution / Alignment
1	SDG 9 - Industry, Innovation and Infrastructure	Develops a kernel-assisted endpoint monitoring prototype for driver security.
2	SDG 16 - Peace, Justice and Strong Institutions	Improves visibility and resilience against privileged endpoint abuse.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

BYOVD mitigation can be viewed through three practical categories relevant to this project: maintained driver blocklists, user-mode endpoint monitoring, and kernel-assisted driver monitoring. Maintained blocklists provide fast identification for previously catalogued drivers. User-mode EDR solutions provide rich process context but can themselves become a target after an attacker gains kernel execution. Kernel-level monitoring provides privileged visibility, but it must be designed carefully because failures can affect the entire operating system. [3], [5], [6], [9]

Table 2.1 Relevant BYOVD Approaches

APPROACH	STRENGTH	LIMITATION
Static driver blocklists	Fast identification of known vulnerable drivers	Depends on database coverage and updates
User-mode EDR monitoring	Rich process and endpoint telemetry	Can be targeted by kernel-level attacks
Kernel driver monitoring	Privileged visibility into driver-load activity	Requires careful callback and memory-safe design
Proposed hybrid framework	Combines driver-load telemetry, hash attestation and self-protection	Current prototype does not claim pre-load driver blocking

2.2 RESEARCH GAPS

- Known-hash databases are precise for previously catalogued driver binaries but do not by themselves solve operational monitoring.
- User-mode monitoring may lose visibility or availability when an attacker uses kernel privileges to interfere with the monitoring service.
- A kernel monitoring agent must avoid blocking sensitive operating-system paths while still maintaining useful telemetry.
- Administrative review should occur outside the kernel callback rather than synchronously inside a privileged notification path.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed framework separates the privileged interception layer from the user-mode analysis and presentation layer. The kernel driver detects system-mode image loads, sends structured driver-load events through a Filter Manager communication port and uses Object Manager callbacks to protect the user-mode service. The service calculates the SHA-256 digest, checks the digest against drivers.json, stores events and exposes a local web dashboard.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- Detect driver-load activity at Ring 0.
- Identify known vulnerable drivers using SHA-256 attestation.
- Provide self-protection for the user-mode analysis service.
- Create auditable event records and analyst-facing visibility.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

- Windows-native architecture using WDK and Filter Manager communication.
- Heavy file I/O and cryptographic hashing remain in user mode.
- A protected service remains available when common administrative termination attempts are made.
- The architecture leaves a clear path for later policy enforcement without placing human interaction in a kernel callback.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system is divided into a kernel-mode interception and protection layer and a user-mode analysis and dashboard layer. When a driver image is mapped, PsSetLoadImageNotifyRoutine provides the kernel event. The event includes the driver path and the callback PID value. For kernel-mode image loads, the observed PID is 0 in the tested environment. The kernel driver also uses ObRegisterCallbacks to reduce dangerous handle rights requested against the protected user-mode service.

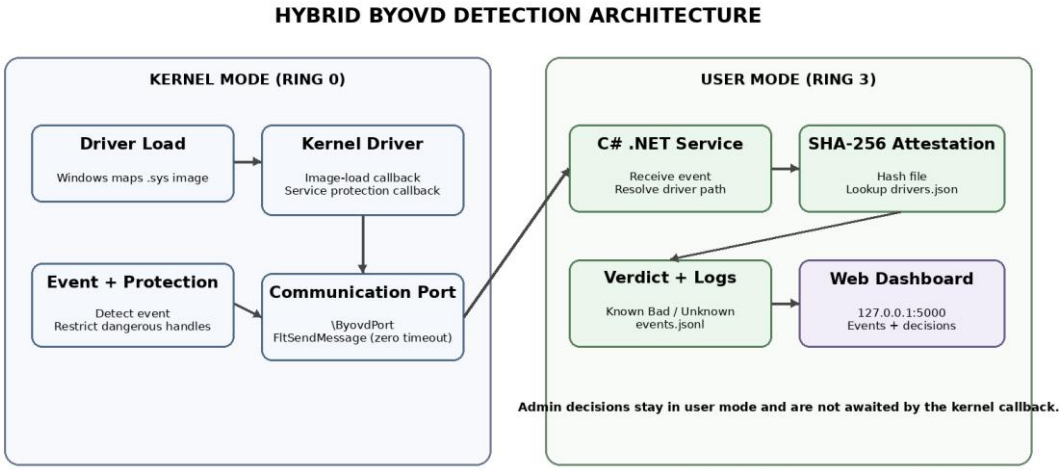


Figure 3.1 Hybrid BYOVD Detection Architecture

3.2 MODULE DESIGN

The implementation is organised into three project modules that correspond to the main security functions demonstrated during testing.

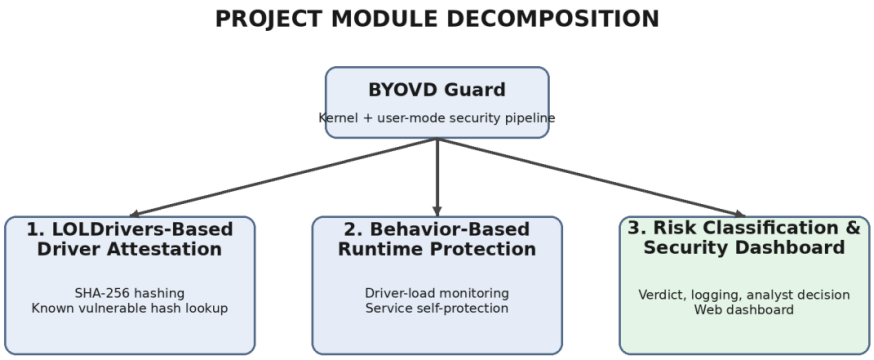


Figure 3.2 Project Module Decomposition

Table 3.1 Project Modules

MODULE	RESPONSIBILITY
LOLDrivers-Based Driver Attestation	Computes SHA-256 for a detected driver image and checks drivers.json for a known vulnerable or test hash.
Behavior-Based Runtime Protection	Uses image-load monitoring and ObRegisterCallbacks to observe driver activity and protect the monitoring service from user-mode termination or memory-write access.
Risk Classification & Security Dashboard	Presents driver paths, hashes and verdicts, stores events in JSON Lines, and records analyst Allow/Block decisions in decisions.json.

3.3 DATA FLOW / UML DIAGRAMS

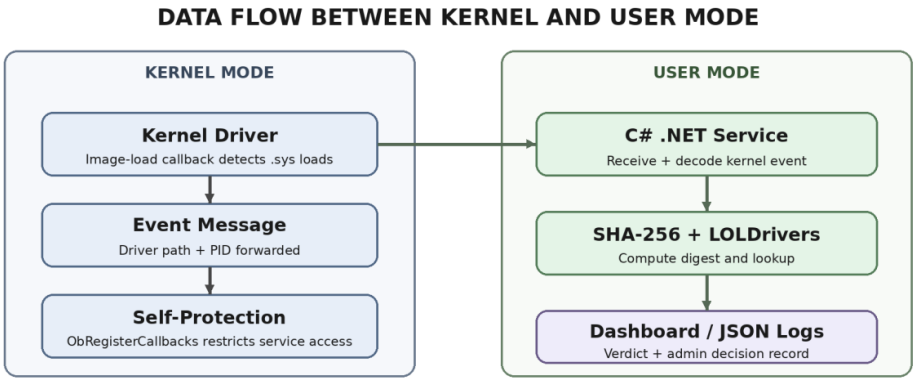


Figure 3.3 Data Flow Between Kernel and User Mode

The kernel component produces the event and the user-mode service consumes it asynchronously. The dashboard communicates with the service rather than directly with Ring 0. This keeps user-interface activity outside the kernel execution path.

3.4 THREAT MODEL AND ATTACK SURFACE

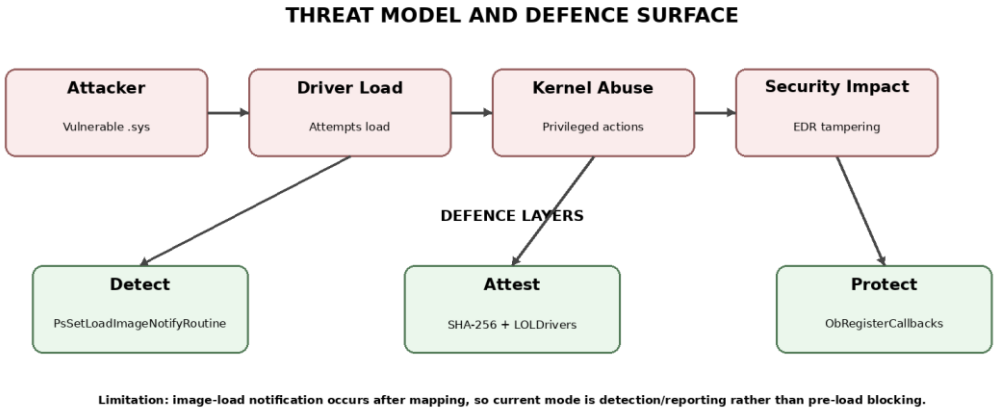


Figure 3.4 Threat Model and Defence Surface

The threat model assumes that an attacker has obtained a legitimate but vulnerable driver and attempts to load it to obtain privileged capabilities or interfere with endpoint security. The principal surfaces are the driver image-load notification path, the communication port, the protected service process, and the driver hash database.

3.5 SECURITY CONTROLS

Table 3.2 Security Controls Against Identified Threats

THREAT	CONTROL
Known vulnerable driver	SHA-256 attestation against the LOLDrivers-derived drivers.json database
User-mode service termination	ObRegisterCallbacks strips PROCESS_ TERMINATE and related rights from targeted handles
Memory-write based service tampering	PROCESS_VM_WRITE and PROCESS_VM_ OPERATION rights are stripped
Unsafe kernel waiting	FltSendMessage uses a zero timeout so callbacks do not wait for the service

CHAPTER 4

SYSTEM DESCRIPTION

4.1 TECHNOLOGY STACK AND TEST ENVIRONMENT

LAYER	TECHNOLOGY / TOOL	PURPOSE
Kernel driver	C, WDK, Filter Manager APIs	Driver load detection, communication and self-protection
Kernel callbacks	PsSetLoadImageNotifyRoutine, ObRegisterCallbacks	Driver-load telemetry and process-handle protection
IPC	FltCreateCommunicationPort / FltSendMessage	Kernel-to-user event delivery
User-mode service	C# / .NET 8	Hashing, database lookup, logging and event handling
Dashboard	ASP.NET Core / Kestrel	Local live monitoring interface
Attestation source	LOLDrivers drivers.json	Known vulnerable-driver hash corpus
Testing	Hyper-V VM, DebugView, Process Explorer, Task Manager, taskkill	Isolated validation of driver, detection and protection

4.2 MODULE IMPLEMENTATION

4.2.1 DRIVER LOAD DETECTION

The kernel driver registers PsSetLoadImageNotifyRoutine and filters the callback using ImageInfo->SystemModeImage. This prevents standard user-mode executable and DLL loads from being treated as driver events. The callback records the driver image path and emits a structured event. During validation, the driver loaded and unloaded cleanly and a Process Explorer kernel driver was detected as PROCEXP152.SYS.

4.2.2 KERNEL-USER COMMUNICATION

The prototype uses a Filter Manager communication port named \ByovdPort. The user-mode client connects through fltlib.dll using P/Invoke. The kernel uses FltSendMessage with a zero timeout, so the image-load callback does not wait for user-mode processing. This is a deliberate safety trade-off: an event can be missed if no client is ready at the instant of transmission.

4.2.3 LOLDRIVERS ATTESTATION

The C# service loads drivers.json at startup and extracts SHA-256 values from the KnownVulnerableSamples entries. During the validated run, the service reported 2,193 loaded hashes. A real driver-load event for PROCEXP152.SYS produced SHA-256 value 487E268A8584561390E2283DBF9F6DFA157F15C2CE401F79 B9424F1560B95FCD and returned Unknown because the exact binary was not in the LOLDrivers-derived list. To verify the positive path, the same hash was deliberately added to extra.txt. On the next run the service reported 2,194 hashes and classified the same event as KNOWN BAD (test).

4.2.4 BEHAVIOR-BASED SELF-PROTECTION

The driver registers an Object Manager pre-operation callback for process handle creation and duplication. When the requested process is the protected service, the callback removes dangerous rights including PROCESS_TERMINATE, PROCESS_CREATE_THREAD, PROCESS_VM_OPERATION, PROCESS_VM_ WRITE and PROCESS_SUSPEND_RESUME. The service itself is exempted through a current-process check.

4.2.5 DASHBOARD AND EVENT LOGGING

The dashboard implementation exposes a local ASP.NET Core page at http://127.0.0.1:5000. The service maintains recent driver events, records events to events.jsonl and persists administrative Allow/Block decisions in decisions.json. The dashboard is deliberately kept in user mode and is not inserted into the synchronous kernel callback path. Final UI validation should be recorded from the final VM run before submission.

4.3 RESULTS AND DISCUSSION

Table 4.1 Implementation Validation Results

TEST	OBSERVED RESULT	STATUS
Kernel driver start	Service reported RUNNING and DebugView displayed "BYOVD Framework Driver Loaded".	PASS
Kernel driver unload/reload	Unloaded message observed; driver started again without crash.	PASS
Driver-load detection	PROCEXP152.SYS detected with path \Device\HarddiskVolume1\Windows\System32\drivers\PROCEXP152.SYS.	PASS

TEST	OBSERVED RESULT	STATUS
Communication port	C# service connected to \ByovdPort and received the driver event.	PASS
LOLDrivers database load	2,193 SHA-256 hashes loaded from drivers.json.	PASS
Positive hash-match path	Adding the observed hash to extra.txt increased the total to 2,194 and returned KNOWN BAD (test).	PASS
Service self-protection	taskkill /F returned Access is denied; Task Manager End task also failed while protection was active.	PASS

The validation confirms the core end-to-end chain: a kernel image-load event is generated, transferred to user mode, hashed, matched against an attestation source and surfaced as a verdict. The positive-path test is intentionally synthetic and is used only to verify that the lookup logic produces a known-bad classification when a matching SHA-256 value is present.

The self-protection test is also deliberately scoped. The demonstrated control prevents common user-mode termination and memory-access requests against the service. A separate vulnerable driver that already has arbitrary kernel write capability could operate through kernel-originated handles and is outside what ObRegisterCallbacks can block in this configuration. The project therefore treats service self-protection as one layer rather than as complete protection against every possible BYOVD primitive.

The use of a zero-timeout FltSendMessage is an additional operational trade-off. It prevents the kernel notification path from waiting on the service, but it means an event may be dropped when no receiver is waiting. A future version can add a bounded in-kernel queue and asynchronous retry semantics without introducing user-mode waits into the callback.

4.4 IMPLEMENTATION VALIDATION

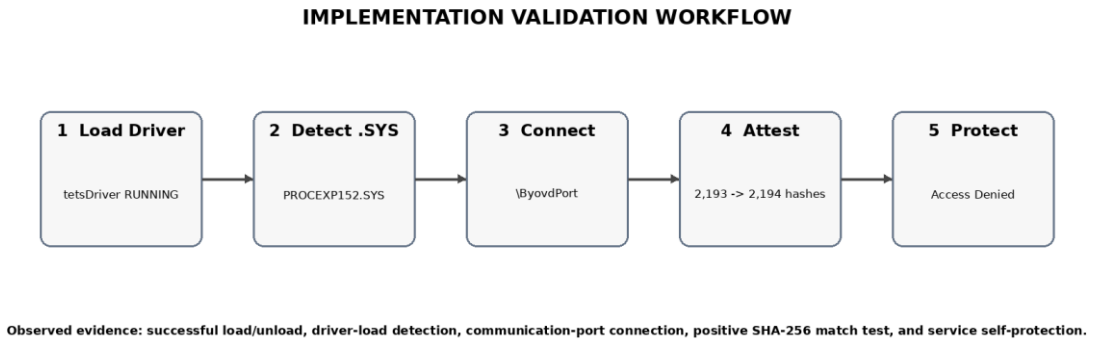


Figure 4.1 Implementation Validation Workflow

The completed Phase I validation therefore supports the following concrete claims: the prototype kernel driver loads and unloads successfully in the test VM; the driver detects system-mode image loads; the communication port connects to the C# service; the service can perform SHA-256 attestation against the loaded database; a controlled positive match produces a KNOWN BAD verdict; and the monitoring service resists administrative termination attempts while the protection callback is active.

CHAPTER 5
CONCLUSION

This project developed a practical Hybrid Attestation and Behavioral Framework for Real-Time BYOVD Detection in Endpoint Protection Platforms. The implementation uses a lightweight Ring-0 driver for kernel-mode driver-load monitoring and self-protection, while a C# .NET service performs file access, SHA-256 attestation and event management in user mode. The architecture is connected through a Filter Manager communication port and provides a local web dashboard for operational visibility.

The demonstrated validation confirmed successful driver startup and clean unload/reload, detection of PROCEXP152.SYS, successful kernel-to-user communication, loading of 2,193 LOLDrivers hashes, a controlled 2,194-hash positive-match test, and protection of the user-mode service from Task Manager and taskkill termination attempts. These results establish the project's core detection, attestation, telemetry and self-protection pipeline without relying on unsupported pre-load blocking or fabricated performance measurements.

The current prototype should be interpreted as a detection and self-protection framework, not as a complete driver prevention product. PsSetLoadImageNotifyRoutine is a notification mechanism invoked after the image is mapped, and the present communication design intentionally avoids blocking the kernel callback while user mode evaluates the event. These constraints are explicit design decisions that provide a safer base for further enforcement research.

FUTURE WORK

- Add a bounded kernel event queue so driver-load messages can be delivered reliably when the service is briefly busy or unavailable.
- Complete end-to-end validation of the dashboard and administrative decision workflow on the final test VM.
- Extend behavioral monitoring to observe and classify access requests directed at protected endpoint-security processes and relevant driver device interfaces.

- Integrate certificate status, driver provenance and Microsoft vulnerable-driver policy data as additional attestation context.
- Investigate a safe pre-load enforcement mechanism such as supported Windows policy controls rather than relying on post-load image notifications.
- Measure CPU, memory, event-throughput and detection latency using a repeatable benchmark procedure after the functional prototype is stable.

REFERENCES

1. R. Gupta, L. Dresel, N. Spahn, G. Vigna, C. Kruegel, and T. Kim, “POPKORN: Popping Windows Kernel Drivers At Scale,” in *Proc. ACSAC*, 2022, pp. 854–868.
2. P. Ciholas, J. M. Such, A. Marnerides, B. Green, J. Zhang, and U. Roedig, “Fast and Furious: Outrunning Windows Kernel Notification Routines from User-Mode,” in *Proc. DIMVA*, 2020, pp. 67–88.
3. J. Vinopal, “Breaking Boundaries: Investigating Vulnerable Drivers and Mitigating Risks,” *Check Point Research*, Sep. 2024.
4. A. Monzani, A. Parata, A. Oliveri, S. Aonzo, D. Balzarotti, and A. Lanzi, “Unveiling BYOVD Threats: Malware’s Use and Abuse of Kernel Drivers,” in *Proc. NDSS*, 2026.
5. Microsoft, “Recommended driver block rules,” *Microsoft Learn Documentation*.
6. A. Klopsch, “Sophos discovers the threat actors behind RansomHub ransomware using EDRKillShifter in attacks,” *Sophos*, Aug. 2024.
7. VMware Carbon Black Threat Analysis Unit, “Hunting Vulnerable Kernel Drivers,” *VMware Security Blog*, Oct. 2023.
8. Microsoft, “Windows Driver Kit (WDK) documentation,” *Microsoft Learn*.
9. LOLDrivers Project, “LOLDrivers: Windows driver research and vulnerable driver database,” *magicsword-io*.

APPENDIX

A PROJECT FILE STRUCTURE

The following files represent the project implementation used in the functional testing described in this report.

PATH / FILE	DESCRIPTION
Driver\driver.c	Kernel driver initialization, image-load monitoring, communication port and self-protection callback.
ByovdService\Program.cs	C# service that connects to \ByovdPort, hashes driver files and performs LOLDrivers lookup.
ByovdDashboard\Program.cs	ASP.NET Core dashboard and service-side event/decision management.
drivers.json	LOLDrivers-derived SHA-256 attestation database used by the service.
extra.txt	Controlled test-hash list used to validate the positive matching path.
events.jsonl	Append-only JSON Lines event log generated by the service.
decisions.json	Persistent Allow/Block decisions recorded by the dashboard.
tets.sys	Prototype kernel-driver build artifact used during VM validation.

B VALIDATION EVIDENCE SUMMARY

Representative evidence from the validated VM run:

- **Kernel driver state:** tetsDriver reported RUNNING after sc start.
- **Kernel event:** DRIVER LOAD DETECTED for PROCEXP152.SYS.
- **IPC:** Connected. Waiting for events... followed by the PROCEXP152.SYS event line.
- **Attestation:** Loaded 2,193 hashes from drivers.json.
- **Controlled positive match:** Loaded 2,194 hashes and returned KNOWN BAD (test).
- **Self-protection:** taskkill /F /IM ByovdService.exe returned Access is denied and Task Manager End task did not terminate the protected service.

The observed PROCEXP152.SYS test hash was 487E268A85845613 90E2283DBF9F6DFA157F15C2CE401F79B9424F1560B95FCD. The synthetic extra.txt entry was used only to demonstrate the positive attestation path and should not be described as evidence that the Process Explorer driver itself is malicious.